

MS&E 228: Applied Causal Inference Powered by ML and AI

Lecture 10: Heterogeneous Treatment Effects

Vasilis Syrgkanis

Stanford University

Winter 2026

Readings: *Applied Causal Inference Powered by ML and AI*, §14.

Goals for Today

1. **Move beyond average effects:** Why is understanding treatment effect heterogeneity crucial, and how do we formalize it through the CATE?
2. **Master the DR-Learner:** How can we extend the doubly robust approach to learn heterogeneous treatment effects for binary treatments?
3. **Master the R-Learner:** How can we extend the DML approach to learn heterogeneous treatment effects for continuous treatments?
4. **Build valid confidence intervals:** How can we construct reliable confidence intervals for CATE estimates, using linear models, Causal Forests, and simultaneous confidence bands?

Part I: Why Heterogeneity Matters

From Average to Conditional Effects

Beyond a Single Number

Up until now, we have focused on estimating average effects (ATE, ATT).

Now we ask: **how does the effect vary across the population?**

Conditional Average Treatment Effect (CATE)

For binary treatments:

$$\tau(x) = \mathbb{E}[Y(1) - Y(0) \mid X = x]$$

The expected treatment effect for individuals with characteristics $X = x$.

For **continuous treatments**, we care about the conditional average marginal effect:

$$\tau(x) = \mathbb{E}\left[\frac{\partial}{\partial d} Y(d) \mid X = x\right]$$

which informs local changes around the current policy.

Why learn a CATE?

What is wrong with Average Effects?

Why CATE Matters: Enhanced Insights

Hidden Heterogeneity

A new intervention might have a statistically zero (or slightly negative) average effect, but there might exist some sub-group of responders. Similarly, a positive-on-average intervention might be harmful for some sub-group.

Examples:

- ▶ A drug we plan to release might harm some subset of patients.
- ▶ A drug we don't plan to release (no stat-sig) might substantially help a sub-group.
- ▶ CATE analysis can help stratify and select participants in a trial to showcase significance.

By averaging, we lose all this information!

Why CATE Matters: Personalized Decision Making

By knowing for which segments the effect is positive and for which it is negative, we can develop **personalized policies**.

- ▶ **A/B Testing:** Most treatments have a negligible average effect. But some subgroup might strictly prefer A and another strictly prefer B. Rather than choosing one, we can do A for one segment and B for the other.
- ▶ **Healthcare:** Use the CATE as a biomarker to decide who to treat with a particular drug.
- ▶ **Operations Management:** Learn which segments of the market are more price sensitive in order to target price discounts.

Case Study 1: Customer Support Interventions

Scenario

A software company wants to understand which customer support intervention (tech support, discount) is more impactful and for which subset of the population, to do better targeting.



Goal: Estimate the CATE of different interventions to personalize customer support.

Reference: EconML – Multi-investment Attribution at A Software Company

Case Study 1: Data

Data Setup ($n = 2,000$ customers, synthetic)

Feature Name	Type	Details
Global Flag	W	whether the customer has global offices
Major Flag	W	whether the customer is a large consumer in their industry
SMC Flag	W	whether the customer is a Small Medium Corporation
Commercial Flag	W	whether the customer's business is commercial (as opposed to public sector)
IT Spend	W	\$ spent on IT-related purchases
Employee Count	W	number of employees
PC Count	W	number of PCs used by the customer
Size	X	customer's size given by their yearly total revenue
Tech Support	T	whether the customer received tech support (binary)
Discount	T	whether the customer was given a discount (binary)
Revenue	Y	\$ Revenue from customer given by the amount of software purchased

Case Study 2: Targeted Price Discounts

Scenario

An online media company wants to understand for which customers they should offer a price discount to improve total revenue.



This boils down to estimating **heterogeneous price elasticities**:

- ▶ Offer discount to segments with elasticity < -1 (elastic demand).
- ▶ Do *not* offer discount to segments with elasticity > -1 (inelastic demand).

Reference: Customer – Segmentation Estimate Individualized Responses to Incentives

This is exactly the scenario from your homework, where you estimate elasticities for various clusters, i.e., $X = \text{cluster}$.

Case Study 2: Data

Data Setup ($n \approx 10,000$ users, synthetic)

Feature Name	Type	Details
account_age	W	user's account age
age	W	user's age
avg_hours	W	the average hours user was online per week in the past
days_visited	W	the average number of days user visited the website per week in the past
friend_count	W	number of friends user connected in the account
has_membership	W	whether the user had membership
is_US	W	whether the user accesses the website from the US
songs_purchased	W	the average songs user purchased per week in the past
income	X	user's income
price	T	the price user was exposed during the discount season (baseline price * small discount)
demand	Y	songs user purchased during the discount season

The Basics

Identification of CATE

Identification of CATE under Conditional Exogeneity

Suppose $Y(d) \perp\!\!\!\perp D \mid Z$ for some controls Z that are a “superset” of X (i.e., X is measurable w.r.t. Z).

Then the CATE is identified by the **conditional g-formula**:

$$\begin{aligned}\tau(X) &:= \mathbb{E}[Y(1) - Y(0) \mid X] \\ &= \mathbb{E}\left[\underbrace{\mathbb{E}[Y \mid D=1, Z]}_{g_0(1,Z)} - \underbrace{\mathbb{E}[Y \mid D=0, Z]}_{g_0(0,Z)} \mid X\right] = \mathbb{E}[g_0(1, Z) - g_0(0, Z) \mid X]\end{aligned}$$

Similarly, for **continuous** treatments the conditional average marginal effect is:

$$\tau(X) := \mathbb{E}\left[\frac{\partial}{\partial d} g_0(D, Z) \mid X\right]$$

Part II: The DR-Learner

CATE Estimation for Binary Treatments

The G-Formula Approach: T-Learner and S-Learner

From the g-formula $\tau(X) = \mathbb{E}[g_0(1, Z) - g_0(0, Z) \mid X]$:

Step 1: Predict Y from D, Z to learn $\hat{g}$.

Step 2: Predict $\hat{g}(1, Z) - \hat{g}(0, Z)$ from X to learn $\hat{\tau}(X)$.

When $Z = X$, Step 2 is unnecessary: simply set $\hat{\tau}(X) = \hat{g}(1, X) - \hat{g}(0, X)$.

- ▶ **T-Learner:** Fit separate models $\hat{g}(1, Z)$ and $\hat{g}(0, Z)$ for treated and control, predicting Y from Z .
- ▶ **S-Learner:** Fit a single model $\hat{g}(D, Z)$ that includes D as a feature.

Poll 1: CATE Identification

Poll Question

If $Y(d) \perp\!\!\!\perp D \mid X$ (i.e., $Z = X$), how does the CATE identification formula simplify?

- A. $\tau(X) = g(1, X) - g(0, X)$
- B. $\tau(X) = \mathbb{E}[Y \mid D=1] - \mathbb{E}[Y \mid D=0]$
- C. It cannot be simplified further
- D. $\tau(X) = \mathbb{E}[Y(1) - Y(0)]$



(Poll Everywhere)

The IPW Approach for CATE

We can also identify the CATE via propensities. Recall the IPW weights

$$H_p(D, Z) = \frac{D}{p(Z)} - \frac{1 - D}{1 - p(Z)}.$$

These weights have the key property that for *any* function g (*prove as exercise*):

$$\mathbb{E}[H_{p_0}(D, Z) \cdot g(D, Z) \mid X] = \mathbb{E}[g(1, Z) - g(0, Z) \mid X]$$

Applying this to the true outcome model g_0 :

$$\mathbb{E}[H_{p_0}(D, Z) \cdot Y \mid X] = \mathbb{E}[H_{p_0}(D, Z) \cdot \underbrace{\mathbb{E}[Y \mid D, Z]}_{g_0(D, Z)} \mid X] = \mathbb{E}[g_0(1, Z) - g_0(0, Z) \mid X] = \tau(X)$$

We can learn CATEs by regressing $H_{p_0}(D, Z) \cdot Y$ on X . This is the **IPW Learner**—but it has very high variance and is almost never used in practice.

Combining: The Doubly Robust Formula for CATE

Similar to the DR estimator for the ATE, we combine the g-formula and IPW:

$$\tau(X) = \mathbb{E} \left[g_0(1, Z) - g_0(0, Z) + H_{p_0}(D, Z) \cdot (Y - g_0(D, Z)) \mid X \right]$$

Define the **doubly robust pseudo-outcome**:

$$Y^{DR}(g, p) = g(1, Z) - g(0, Z) + H_p(D, Z) \cdot (Y - g(D, Z))$$

Key Property

$\tau(X) = \mathbb{E}[Y^{DR}(g_0, p_0) \mid X]$. This is **doubly robust**: it gives the correct CATE if either the propensity model or the outcome model is correct.

The DR-Learner Algorithm

Algorithm (DR-Learner with Cross-Fitting)

1. Train $\hat{g}$ to predict Y from D, Z (cross-fitted).
2. Train $\hat{p}$ to predict D from Z (cross-fitted).
3. Construct pseudo-outcomes:
$$Y_i^{DR} = \hat{g}(1, Z_i) - \hat{g}(0, Z_i) + H_{\hat{p}}(D_i, Z_i)(Y_i - \hat{g}(D_i, Z_i)).$$
4. Train a final model $\hat{\tau}$ to predict Y^{DR} from X .

```
# Steps 1-3: cross-fitted nuisance models and pseudo-outcomes
Y_dr = g1_hat - g0_hat + H_p_hat * (Y - gd_hat)

# Step 4: Train final CATE model
cate_model = AutoML().fit(X, Y_dr)
tau_hat = cate_model.predict(X_test)
```

DR-Learner with EconML

```
from econml.dr import DRLearner
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.ensemble import GradientBoostingClassifier

# Generic DR-Learner with a flexible final model
dr = DRLearner(
    model_regression=GradientBoostingRegressor(),
    model_propensity=GradientBoostingClassifier(),
    model_final=GradientBoostingRegressor()
)
dr.fit(Y, D, X=X, W=Z)

# Get CATE predictions
tau_hat = dr.effect(X_test)
```

Part III: Confidence Intervals for CATE

Linear Models, Causal Forests, and Confidence Bands

The Problem with Generic ML Final Models

What if we want confidence intervals for our CATE estimates?

The generic DR-Learner uses any ML model in the final step:

$$\hat{\tau}(X) = \text{AnyMLModel}(X)$$

Even if we ignore first-stage errors for $\hat{g}$, $\hat{p}$, we still don't know how to produce confidence intervals for a generic ML predictor!

Challenge

We need to restrict the final model class to get valid inference.

Solution 1: Linear Final Model (Best Linear Projection)

Restrict the final model to be **linear** in some features $\phi(X)$:

$$\min_{\beta} \mathbb{E} \left[(Y^{DR}(\hat{g}, \hat{p}) - \beta' \phi(X))^2 \right]$$

This is a simple multivariate OLS: $Y^{DR} \sim \phi(X)$.

Best Linear Projection

Even if the true CATE is nonlinear, this gives the **best linear projection**:

$$\begin{aligned} \min_{\beta} \mathbb{E}[(Y^{DR} - \beta' \phi(X))^2] &= \min_{\beta} \mathbb{E}[(\mathbb{E}[Y^{DR}|X] - \beta' \phi(X))^2] \\ &= \min_{\beta} \mathbb{E}[(\tau(X) - \beta' \phi(X))^2] \end{aligned}$$

Asymptotic Normality for the Linear DR-Learner

Theorem (Asymptotic Normality)

Under the product rate condition on nuisances ($\sqrt{n} \cdot \text{RMSE}(\hat{g}) \cdot \text{RMSE}(\hat{p}) \rightarrow 0$) and consistency of both models ($\text{RMSE}(\hat{g}), \text{RMSE}(\hat{p}) \rightarrow 0$), and other mild regularity assumptions:

$$\sqrt{n}(\hat{\beta} - \beta_0) \xrightarrow{d} N(0, V)$$

where V is the sandwich variance:

$$V = (\mathbb{E}[\phi(X)\phi(X)'])^{-1} \mathbb{E}[\phi(X)\phi(X)'\epsilon^2] (\mathbb{E}[\phi(X)\phi(X)'])^{-1}$$

with $\epsilon = Y^{DR} - \beta_0' \phi(X)$.

The HC0 covariance matrix from OLS packages estimate exactly V/n . We can use heteroskedasticity-robust standard errors for $\hat{\beta}$ and CATE predictions $\hat{\tau}(x) = \hat{\beta}' \phi(x)$.

Linear DR-Learner with EconML

```
from econml.dr import LinearDRLearner

# Linear final model => confidence intervals!
ldr = LinearDRLearner(
    model_regression=GradientBoostingRegressor(),
    model_propensity=GradientBoostingClassifier()
)
ldr.fit(Y, D, X=X, W=Z)

# Point estimates and confidence intervals
tau_hat = ldr.effect(X_test)
lb, ub = ldr.effect_interval(X_test, alpha=0.05)

# Inspect coefficients
print(ldr.summary())
```

A Parenthesis: Simultaneous Confidence Bands

When we look at many coefficients β_j or many predictions $\hat{\tau}(x_1), \dots, \hat{\tau}(x_K)$, we run into the **multiple testing** trap.

Each individual 95% CI is correct *if we had fixed the target quantity before looking at the data*. But if we look at results and focus on the “interesting” ones, these CIs are not valid!

Compounding Failure

With K target points, the probability that at least one CI fails to contain the true value can be much larger than 5%.

Solution: We want **confidence bands**—intervals $CI_{x_i} = [L_{x_i}, U_{x_i}]$ such that:

$$\Pr(\forall x \in \{x_1, \dots, x_K\} : \tau(x) \in CI_x) \approx 95\%$$

How to Construct Confidence Bands with Linear Models

The estimated predictions are approximately **jointly Gaussian** (if $\log(K) \ll n$):

$$\{\sqrt{n}(\hat{\tau}(x_i) - \tau(x_i))\}_{i=1}^K \stackrel{a}{\sim} N(0, \Sigma), \quad \Sigma_{ij} = x_i' V x_j$$

We want bands of the form $CI_{x_i} = \hat{\tau}(x_i) \pm c \cdot \sqrt{\Sigma_{ii}/n}$. Then:

$$\Pr(\text{all covered}) = \Pr\left(\max_i \frac{\sqrt{n}|\hat{\tau}(x_i) - \tau(x_i)|}{\sqrt{\Sigma_{ii}}} \leq c\right)$$

Finding c by Simulation

1. Draw many samples from $N(0, D^{-1/2}\Sigma D^{-1/2})$ where $D = \text{diag}(\Sigma_{11}, \dots, \Sigma_{KK})$.
2. For each sample, compute the max absolute coordinate value.
3. Take the 95th percentile of these maxima, across samples. That is c .

Confidence Bands: Pseudocode (1/2)

Step 1–3: Fit the linear final model with HCO standard errors and compute the joint covariance of CATE predictions at test points.

```
import statsmodels.api as sm
import numpy as np

# 1. Fit final linear model on DR pseudo-outcomes
final_model = sm.OLS(Y_dr, X_final).fit(cov_type='HCO')
# 2. Predictions & joint covariance at test points
#     V = HCO covariance of beta_hat
#     Sigma_ij = x_i' V x_j
preds = final_model.predict(X_test)
V = final_model.cov_params() # HCO covariance of beta
Sigma = X_test @ V @ X_test.T # joint covariance of tau_hat
# 3. Standardize to correlation matrix
std_devs = np.sqrt(np.diag(Sigma))
D_inv = np.diag(1 / std_devs)
corr = D_inv @ Sigma @ D_inv
```

Confidence Bands: Pseudocode (2/2)

Step 4–5: Simulate the critical value c and construct the bands.

```
# 4. Simulate to find the critical value c
samples = np.random.multivariate_normal(
    np.zeros(len(X_test)), corr, size=5000)
max_abs = np.max(np.abs(samples), axis=1)
c = np.percentile(max_abs, 95)

# 5. Simultaneous confidence bands
lower = preds - c * std_devs
upper = preds + c * std_devs
```

Key Insight

The critical value c replaces 1.96 from pointwise CIs. Because $c > 1.96$, the simultaneous bands are **wider** than pointwise CIs, but they guarantee 95% joint coverage across all test points.

Solution 2: Causal Forests

What if we want a more flexible model than linear, but still need confidence intervals?

Doubly Robust Causal Forest

A Causal Forest learns a random forest to solve $Y^{DR} \sim X$, with crucial alterations:

1. **Sub-sampling without replacement:** Each tree uses a sub-sample of size $s \ll n$.
2. **Honesty:** Half of the sub-sample determines the tree structure (splits); the other half populates the leaves with predictions.
3. **Balanced splits:** At least some constant fraction of samples falls on each side. With small probability, a random feature is chosen.

Under these conditions, predictions are **asymptotically normal** and standard errors can be computed via a bootstrap-style method.

Causal Forests in Practice

Implemented in the **GRF** package (R) and the **grf** module in **EconML** (Python).

```
from econml.dr import ForestDRLearner

# DR-Learner with a Causal Forest final model
cf = ForestDRLearner(
    model_regression=GradientBoostingRegressor(),
    model_propensity=GradientBoostingClassifier()
)
cf.fit(Y, D, X=X, W=Z)

# Point estimates with confidence intervals
tau_hat = cf.effect(X_test)
lb, ub = cf.effect_interval(X_test, alpha=0.05)
```

Note

These are **pointwise** CIs, not simultaneous bands. Uniform bands for causal forests are a very recent development.

Poll 2: Choosing a CATE Estimator

Poll Question

You have a binary treatment and want to estimate CATE with valid confidence intervals. Which approach would you choose?

- A. T-Learner with a random forest
- B. DR-Learner with a neural network final model
- C. LinearDRLearner (OLS final model)
- D. Both C and a ForestDRLearner



(Poll Everywhere)

Revisiting Case Study 1

DRLearner for Understanding ROI

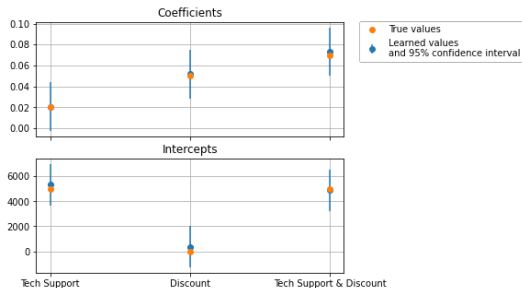
Case Study 1: Fitting the 'LinearDRLearner'

We have multiple discrete treatments, so we encode them as integers $T \in \{0, 1, 2, 3\}$. We use 'LinearDRLearner' assuming the CATE is linear in 'Size'.

```
# Train EconML model with generic helper models
model = LinearDRLearner(
    model_regression=XGBRegressor(learning_rate=0.1, max_depth=3),
    model_propensity=XGBClassifier(learning_rate=0.1, max_depth=3,
        objective="multi:softprob"),
    random_state=1,
)
# Specify final stage inference type and fit model
model.fit(Y=Y, T=T, X=X, W=W, inference="statsmodels")

# Get summary for each treatment
for i in range(model._d_t[0]):
    display(model.summary(T=i + 1))
```

Case Study 1: Interpreting the CATE

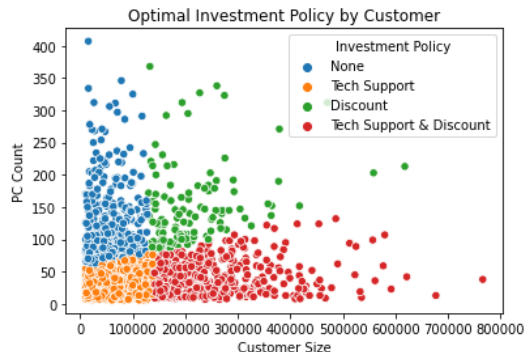


Interpretation:

- ▶ The model successfully recovers the true coefficients and intercepts within the 95
- ▶ **Tech Support:** High base effect (intercept $\approx$ \$5000), small positive slope on 'Size'.
- ▶ **Discount:** No base effect, but a stronger positive slope on 'Size'.
- ▶ **Both:** Additive effect, combining the high base of tech support with the strong slope of the discount.

Case Study 1: Optimal Policy

After estimating the CATE, we can determine the optimal, profit-maximizing policy for each customer, accounting for treatment costs.



Policy Rules:

- ▶ Small customers ('Size' low): No incentive.
- ▶ Small customers with few PCs: Tech support.
- ▶ Medium-to-large customers: Discount.
- ▶ Large customers with few PCs: Both incentives.

Part IV: The R-Learner

CATE Estimation for Continuous Treatments

From DML to the R-Learner

For continuous treatments, we use the DML analogue. Assume a partially linear model with heterogeneous effects:

$$\mathbb{E}[Y(d) \mid Z] = \tau(X) d + f(Z)$$

Under conditional exogeneity, this gives us:

$$\mathbb{E}[Y \mid D, Z] = \tau(X) D + f(Z)$$

Following the DML residualization logic:

$$\begin{aligned} Y - \mathbb{E}[Y|Z] &= \tau(X) D + f(Z) + \epsilon - \tau(X) \mathbb{E}[D|Z] - f(Z) \\ &= \tau(X) (D - \mathbb{E}[D|Z]) + \epsilon \end{aligned}$$

Denoting residuals $\tilde{Y} = Y - \mathbb{E}[Y|Z]$ and $\tilde{D} = D - \mathbb{E}[D|Z]$:

$$\tilde{Y} = \tau(X) \tilde{D} + \epsilon$$

The R-Learner Objective

We can learn $\tau(X)$ by minimizing the squared loss:

$$\min_{\tau(\cdot)} \mathbb{E} \left[(\tilde{Y} - \tau(X) \tilde{D})^2 \right]$$

Note that this is equivalent to:

$$\min_{\tau(\cdot)} \mathbb{E} \left[\left(\frac{\tilde{Y}}{\tilde{D}} - \tau(X) \right)^2 \tilde{D}^2 \right]$$

This is a **sample-weighted regression** of the ratio $\tilde{Y}/\tilde{D}$ on X , with sample weights $\tilde{D}^2$.

The R-Learner Algorithm

Algorithm (R-Learner with Cross-Fitting)

1. Train $\hat{h}(Z)$ to predict Y from Z (cross-fitted).
2. Train $\hat{p}(Z)$ to predict D from Z (cross-fitted).
3. Compute residuals: $\check{Y}_i = Y_i - \hat{h}(Z_i)$, $\check{D}_i = D_i - \hat{p}(Z_i)$.
4. Train $\hat{\tau}$ to predict $\check{Y}/\check{D}$ from X with weights $\check{D}^2$.

```
# Steps 1-3: cross-fitted residuals (same as DML)
Y_res = Y - h_hat_cf      # cross-fitted Y residuals
D_res = D - p_hat_cf      # cross-fitted D residuals

# Step 4: weighted regression
cate_model = AutoML()
cate_model.fit(X, Y_res / D_res, sample_weight=D_res**2)
tau_hat = cate_model.predict(X_test)
```


The R-Learner Algorithm with FLAML AutoML

Targeting the Weighted Square Loss with FLAML

In order to optimize the sample-weighted square loss:

$$\frac{1}{n} \sum_{i=1}^n \check{D}_i^2 \cdot (\check{Y}_i / \check{D}_i - \tau(X))^2$$

we need to define an appropriate custom weighted mse metric.

Default weighted MSE metrics typically "normalize the weights", i.e. optimize

$$\sum_i w_i (Y_i - f(X_i))^2 / \sum_i w_i,$$

which can be slightly inaccurate here (e.g. when performing cross-validation).

The R-Learner Algorithm with FLAML AutoML

```
import numpy as np
from flaml import AutoML

def weighted_sse(
    X_val, y_val, estimator, labels,
    X_train, y_train,
    weight_val=None, weight_train=None,
    *args):
    """Un-normalized weighted sum of squared errors on validation set."""
    y_pred = estimator.predict(X_val)
    if weight_val is not None:
        loss = np.mean(weight_val * (y_val - y_pred) ** 2)
    else:
        loss = np.mean((y_val - y_pred) ** 2)
    return loss, {"weighted_sse": loss}

automl = AutoML(task="regression", eval_method='cv',
                metric=weighted_sse, # custom weighted metric (un-normalized)
                time_budget=60)
automl.fit(X_train=X, y_train=Y_res / D_res, sample_weight=D_res**2)
```

R-Learner with Linear and Forest Final Models

The R-Learner can be combined with different final models:

- ▶ **LinearDML** (linear final model): Estimates a linear CATE $\tau(X) = \beta' \phi(X)$ with valid confidence intervals via OLS standard errors.
- ▶ **CausalForestDML** (forest final model): Estimates a flexible CATE with pointwise confidence intervals via the honest forest methodology.

R-Learner with Linear and Forest Final Models in EconML

```
from econml.dml import LinearDML, CausalForestDML

# Linear CATE model with confidence intervals
ldml = LinearDML(model_y=LGBM(), model_t=LGBM())
ldml.fit(Y, D, X=X, W=Z)
print(ldml.summary())
tau_hat = ldml.effect(X_test)
lb, ub = ldml.effect_interval(X_test, alpha=0.05)

# Flexible CATE model with confidence intervals
cfdml = CausalForestDML(model_y=LGBM(), model_t=LGBM())
cfdml.fit(Y, D, X=X, W=Z)
tau_hat = cfdml.effect(X_test)
lb, ub = cfdml.effect_interval(X_test, alpha=0.05)
```

What Does the R-Learner Estimate Under Misspecification?

If true model is $\mathbb{E}[Y(d) \mid Z] = \theta(Z) D + f(Z)$ (CATE depends on all of Z):

$$\begin{aligned}\min_{\tau} \mathbb{E}[(\tilde{Y} - \tau(X)\tilde{D})^2] &= \min_{\tau} \mathbb{E}[(\theta(Z)\tilde{D} - \tau(X)\tilde{D})^2] \\ &= \min_{\tau} \mathbb{E}[(\theta(Z) - \tau(X))^2 \text{Var}(D|Z)]\end{aligned}$$

Overlap-Weighted Approximation

R-Learner finds the best **overlap-weighted** approximation of the true CATE, giving more weight to regions where $\text{Var}(D|Z)$ is large (treatment more “random”).

Analogous to how DML for the ATE estimates an overlap-weighted average effect.

Generalized PLM with Heterogeneous Effects

What if the treatment effect is linear in some features of the treatment, $\phi(D)$?

$$\mathbb{E}[Y \mid D, Z] = \tau(X)' \phi(D) + f(Z)$$

where $\tau(X)$ is now a vector of CATEs, one for each treatment feature.

Residualizing gives: $\check{Y} = \tau(X)' \check{\phi}(D) + \epsilon$, where $\check{\phi}(D) = \phi(D) - \mathbb{E}[\phi(D)|Z]$.

Algorithm (Generalized R-Learner)

1. Train models to predict Y and each $\phi_j(D)$ from Z (cross-fitted).
2. Compute estimated residuals $\check{Y}$ and $\check{\phi}_j(D)$ for each treatment feature j .
3. Minimize square loss: $\min_{\tau} \mathbb{E}_n[(\check{Y} - \tau(X)' \check{\phi}(D))^2]$

Handling Complex Treatments with `treatment_featurizer`

EconML makes this easy with the `treatment_featurizer` argument.

```
from econml.dml import LinearDML
from sklearn.preprocessing import PolynomialFeatures

# Create polynomial features of the treatment
# D -> [1, D, D^2, ...]
featurizer = PolynomialFeatures(degree=2, include_bias=True)

# DML with featurized treatment
ldml = LinearDML(model_y=LGBM(), model_t=LGBM(),
                 treatment_featurizer=featurizer)

# fit() will automatically handle the featurization
ldml.fit(Y, D, X=X, W=Z)

tau_hat = ldml.effect(X_test)
```

Revisiting Case Study 2

DML for Heterogeneous Price Elasticities ROI

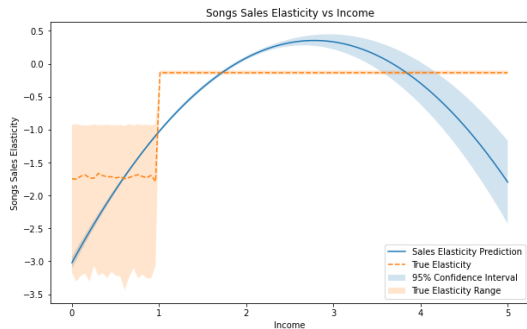
Case Study 2: 'LinearDML' vs 'CausalForestDML'

We fit two models to estimate the price elasticity, heterogeneous in 'income'.

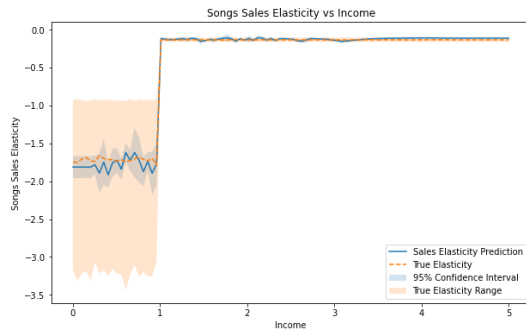
```
# Linear treatment effect heterogeneity, with polynomial features
est_linear = LinearDML(
    model_y=GBR(),
    model_t=GBR(),
    featurizer=PolynomialFeatures(degree=2))
est_linear.fit(log_Y, log_T, X=X, W=W)

# Flexible treatment effect heterogeneity with Causal Forest
est_forest = CausalForestDML(
    model_y=GBR(),
    model_t=GBR())
est_forest.fit(log_Y, log_T, X=X, W=W)
```

Case Study 2: Comparing Model Fits



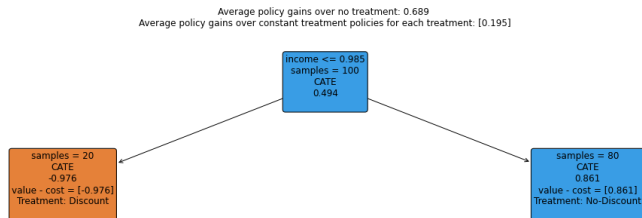
'LinearDML' imposes a quadratic structure, missing the sharp drop.



'CausalForestDML' is flexible and captures the non-linear step function well.

Case Study 2: Policy Interpretation

We can use 'SingleTreePolicyInterpreter' to learn a simple, interpretable policy tree from the complex Causal Forest model.



Learned Policy:

- ▶ If 'income' ≤ 0.985 : Offer the discount (elastic segment).
- ▶ If 'income' > 0.985 : Do not offer the discount (inelastic segment).

This simple rule, derived from a complex model, allows the company to easily target its price discounts to maximize revenue.

Summary: Four Key Takeaways

1. **CATE Unlocks Personalization:** Moving beyond average effects to the Conditional Average Treatment Effect $\tau(x) = \mathbb{E}[Y(1) - Y(0)|X=x]$ is crucial for personalized policies, enhanced data insights, and discovering hidden heterogeneity.
2. **DR-Learner for Binary Treatments:** The DR-Learner constructs a doubly robust pseudo-outcome Y^{DR} and regresses it on X . It is doubly robust and can use any ML model for the final step.
3. **R-Learner for Continuous Treatments:** The R-Learner extends DML to heterogeneous effects by solving a weighted residual-on-residual regression, with weights $\tilde{D}^2$.
4. **Inference via Linear Models or Causal Forests:** Valid CIs come from a linear final model (best linear projection + OLS SEs) or Causal Forests (honest, sub-sampled trees + bootstrap SEs). For multiple comparisons, use simultaneous confidence bands.

Next Time

- ▶ What if we want to be even more flexible with the final CATE model?
- ▶ How should we deal with the fact that generic ML models don't provide confidence intervals?
- ▶ How should we select among CATE models and statistically validate the learned model?

References

- ▶ Nie, X., & Wager, S. (2021). Quasi-oracle estimation of heterogeneous treatment effects. *Biometrika*, 108(2), 299–319.
- ▶ Foster, D. J., & Syrgkanis, V. (2023). Orthogonal statistical learning. *The Annals of Statistics*, 51(3), 879–908.
- ▶ Kennedy, E. H. (2023). Towards optimal doubly robust estimation of heterogeneous causal effects. *Electronic Journal of Statistics*, 17(2), 3008–3049.
- ▶ Athey, S., Tibshirani, J., & Wager, S. (2019). Generalized random forests. *The Annals of Statistics*, 47(2), 1148–1178.
- ▶ Ritzwoller, D. M., & Syrgkanis, V. (2024). Simultaneous Inference for Local Structural Parameters with Random Forests. arXiv preprint arXiv:2405.07860.
- ▶ Semenova, V., & Chernozhukov, V. (2021). Debiased machine learning of conditional average treatment effects and other causal functions. *The Econometrics Journal*, 24(2), 264–289.
- ▶ EconML Documentation: <https://econml.azurewebsites.net/>